TASK: 26

Static & Dynamic Analysis (Mobile VAPT)

Gabbani Surendra

What is Static analysis ?

Static analysis means analyzing an Android app without running it.
You inspect the APK's code, files, permissions, configurations to find security issues.

What we analyze:

1. Decompile APK

- Tools: jadx, apktool, MobSF, JD-GUI
- Check source code & resources.

2. Analyze Manifest File (AndroidManifest.xml)

- Dangerous permissions (e.g., READ_SMS, WRITE_SETTINGS)
- Exported activities, services, receivers
- AllowBackup = true
- Debuggable = true

3. Review Hard-coded Secrets

- API keys, tokens, credentials
- Hardcoded URLs or IPs

4. Look for Insecure Code Practices

- Weak crypto (MD5, SHA1, ECB mode)
- Insecure storage (SharedPreferences, SQLite without encryption)
- Logging sensitive data

5. Analyze Network Security

- HTTP instead of HTTPS
- Bypassable SSL pinning
- Custom Hostname Verifier issues

6. Identify Third-party Libraries

- Outdated or vulnerable libraries
- Using tools like Wappalyzer, MobSF scan results

Static Analysis Tools:

- MobSF (Mobile Security Framework)
- APKTool
- Jadx
- QARK

Simple Visual Diagram:

```
┌──────────────────────────┐
│   APK File (.apk)   │
└──────────────┬───────────┘
               │
       ┌───────┴───────┐
       │               │
       ▼               ▼
┌──────────┐     ┌──────────────┐
│ STATIC ANALYSIS │   │ DYNAMIC ANALYSIS │
└──────────┘     └──────────────┘
```

- Decompile APK
- Review code
- Manifest analysis
- Hardcoded keys
- Permissions check
- Crypto review

- Run app on device
- Monitor traffic
- Hook functions (Frida)
- Bypass security
- Inspect logs/storage
- Runtime behavior

What is Dynamic Analysis?

Dynamic analysis means analyzing the app while it is running on an emulator or device. You test app behavior, runtime security, network traffic, and backend interactions.

What we Analysis?

1. Run the App on Emulator/Device

- Observe runtime behavior, errors, logs.

2. Intercept Network Traffic

Using Burp Suite, MITMProxy, Charles Proxy

Check for:

- HTTP traffic
- Weak TLS
- Leaking tokens
- Insecure API calls

3. Bypass SSL Pinning

- Using Frida, Objection, Magisk environment

4.Test Runtime Permissions

- Check if app requests unnecessary permissions
- Check if it crashes when denied

5.Analyze Data Storage at Runtime

- SharedPreferences
- Databases
- Cache files
- Local storage leaks

6.Runtime Code Injection / Hooking

- Using Frida scripts
- Modify function behavior
- Bypass authentication or root detection

7.Check Memory for Sensitive Data

- Using Memory dump tools
- Check tokens/credentials in RAM

8.Perform Device-Level Tests

- Root detection bypass
- Emulator detection bypass
- Debugger checks

Dynamic Analysis Tools:

- Burp Suite
- Frida
- Objection
- MobSF Dynamic Analyzer
- Xposed Framework
- Wireshark
- ADB commands

THANK YOU